

LiveCaption

Speaker-Attributed Live Captioning System

Technical Documentation

Implementation Details | Stages 1-5

Platform: macOS 14.0+ (SwiftUI + AVFoundation + Vision + Accelerate)

Language: Swift 5 | Build: XcodeGen + Xcode

ML Frameworks: Apple Vision, WhisperKit (CoreML), vDSP (Accelerate)

AIML Institute - Challenge 1: Audio Auto-Captioning

June 2026

Table of Contents

1. System Overview
2. Architecture & Pipeline Diagram
3. Stage 1 - Audio-Visual Capture & Face Detection
4. Stage 2 - Active Speaker Detection (LAR + Differential Motion)
5. Stage 3 - Video-Guided Spectral Separation
6. Stage 4 - Speech Transcription (WhisperKit)
7. Stage 5 - Speaker Attribution (Word-to-Face Mapping)
8. Offline Analysis Lab (Analyze File Mode)
9. Current Limitations & Known Issues
10. Improvement Opportunities
11. Performance Evaluation & Comparison Ideas
12. Source File Reference

1. System Overview

What is LiveCaption?

LiveCaption is a macOS desktop application that provides real-time, speaker-attributed captioning from a single camera and microphone. It watches a video feed, detects who is talking by analyzing lip movements, and transcribes their speech using an on-device AI model (WhisperKit with CoreML). Each transcribed line is labeled with the speaker's identity (e.g., Speaker 1, Speaker 2), creating a full conversation transcript with speaker turns.

Key Features

- Real-time face detection and landmark tracking (Apple Vision framework)
- Active speaker detection via lip motion analysis (no audio diarization needed)
- Cross-modal overlap detection: combines visual (lip) and audio (VAD) cues
- Video-guided spectral separation for overlapping speech (STFT + harmonic masking)
- On-device Indonesian speech transcription (WhisperKit, CoreML + Neural Engine)
- Word-level speaker attribution using video timeline alignment
- Offline analysis lab for evaluating separation quality on recorded files
- Fully on-device processing - no cloud APIs, works offline after model download

Technology Stack

Platform: macOS 14.0+ (Sonoma)

Language: Swift 5.0 (SwiftUI + AppKit)

Build System: XcodeGen (project.yml) + Xcode

Face Detection: Apple Vision (VNDetectFaceLandmarksRequest Rev3)

Audio Capture: AVFoundation (AVCaptureSession, shared video+audio)

Signal Processing: Accelerate / vDSP (STFT, FFT, vector math)

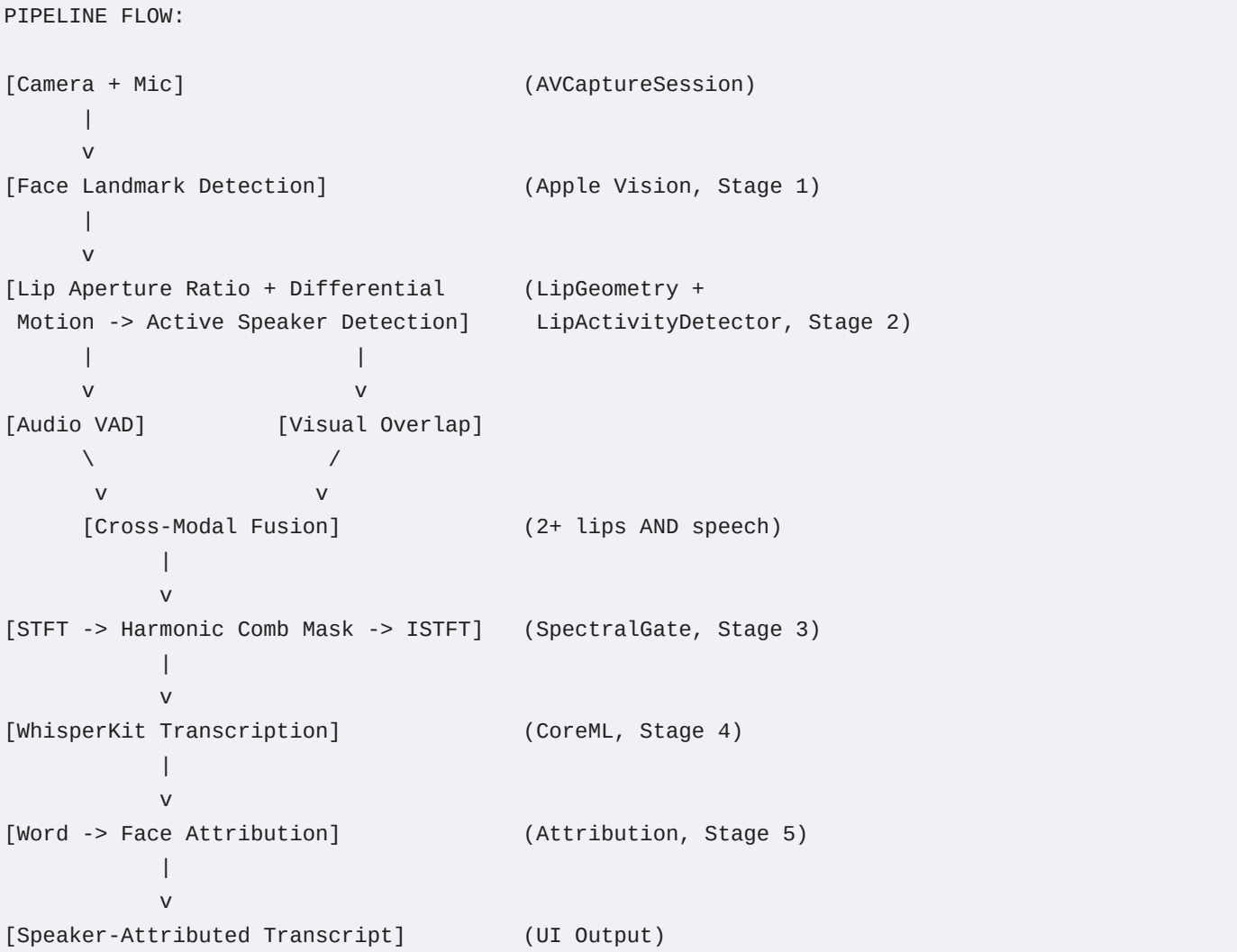
Transcription: WhisperKit 0.18.0 (CoreML + Neural Engine)

ASR Model: openai_whisper-small (multilingual, Indonesian)

2. Architecture & Pipeline Diagram

End-to-End Pipeline

The system processes data through five sequential stages, each building on the previous. The pipeline runs in two modes: Live (real-time from camera/mic) and Analyze File (offline on recorded video). The offline mode runs stages 1-5 in batch; the live mode currently runs stages 1-2 in real-time with stages 3-5 available in the analysis lab.



Data Flow Summary

1. AVCaptureSession delivers video frames and audio buffers simultaneously.
2. Each video frame goes through Vision face landmark detection.
3. Lip landmarks are analyzed for motion (differential: lip minus head movement).
4. Audio is analyzed with an energy-based VAD (adaptive noise floor).
5. Overlap = (2+ faces with active lips) AND (audio VAD detects speech).
6. When overlap is detected, spectral masking isolates the target speaker's harmonics.

-
7. The (optionally separated) audio is transcribed by WhisperKit.
 8. Each transcribed word is assigned to the dominant active face at that timestamp.
 9. Consecutive words from the same speaker merge into utterances.

3. Stage 1 - Audio-Visual Capture & Face Detection

Purpose

Capture synchronized video and audio from the device's camera and microphone, and detect faces with full landmark geometry in each video frame. This is the foundation that all later stages build upon.

How It Works

A. Unified AVCaptureSession

A single AVCaptureSession handles both video and audio capture. This is a deliberate design choice - an earlier version used a separate AVAudioEngine for audio, but it failed to deliver buffers when an external/Continuity camera was connected (the camera hijacked the audio route). By using one session for both, audio always flows alongside video.

The session runs on a dedicated serial queue (sessionQueue). Video frames are processed on videoQueue, audio buffers on audioQueue. Both share the same AVCaptureOutput delegate, with routing determined by checking which output produced the sample buffer.

B. Face Landmark Detection

Each video frame (as a CVPixelBuffer) is passed to Apple Vision's VNDetectFaceLandmarksRequest (Revision 3) via a VNSequenceRequestHandler. This returns VNFaceObservation objects containing:

- Bounding box: normalized rectangle of the face in the image
- 68 facial landmarks grouped into regions: outerLips, innerLips, leftEye, rightEye, nose, noseCrest, leftEyebrow, rightEyebrow, faceContour, medianLine, pupils
- All landmark points are normalized within the face bounding box (0-1 range)

The detection runs synchronously on the video processing queue. Vision's Revision 3 is the most accurate landmark model available on macOS 14, running efficiently on the Neural Engine.

C. Camera Preview with Overlays

CameraPreview wraps an AVCaptureVideoPreviewLayer in an NSView (via NSViewRepresentable for SwiftUI). On top, CAShapeLayer overlays draw face bounding boxes and landmark contours. Active speakers get green outlines (thick), inactive get gray (thin). Each face has a floating 'S1 . 0.15' label showing the speaker ID and current LAR value.

Important: preview mirroring is disabled so that Vision's coordinate space matches the on-screen display. Without this, face overlay boxes would appear flipped.

D. Audio Metrics

AudioMetrics reads raw PCM data directly from CMSampleBuffer - it handles both Float32 and Int16 formats. For Float32, it uses vDSP_measqv (hardware-accelerated mean-square). This gives a single energy value per audio buffer that feeds the Voice Activity Detector.

Source Files

[CaptureManager.swift](#)

Unified session for video + audio, delegates, lifecycle

[FaceLandmarkProcessor.swift](#)

Wraps VNDetectFaceLandmarksRequest (Rev3)

[CameraPreview.swift](#)

NSViewRepresentable + overlay CAShapeLayers

[AudioMetrics.swift](#)

CMSampleBuffer -> mean-square energy (vDSP)

[LiveCaptionApp.swift](#)

App entry point, window configuration

[ContentView.swift](#)

Live/Analyze mode switch, owns CaptureManager

4. Stage 2 - Active Speaker Detection

Purpose

Determine which detected faces are actively speaking at any moment. This uses two independent signals - visual lip motion and audio energy - fused together to reduce false positives.

How It Works

A. Lip Aperture Ratio (LAR)

For each detected face, the Lip Aperture Ratio is computed as:

$$\text{LAR} = \text{vertical_extent}(\text{innerLips}) / \text{horizontal_extent}(\text{outerLips})$$

This is the inner lip opening height divided by the mouth width. The ratio is:

- Scale-invariant: computed in pixel space, so it works regardless of face distance
- Near 0 when the mouth is closed
- Rises toward 0.3-0.5 when speaking

LAR is used for display only (the 'LAR' meter in the sidebar). The actual activity detection uses differential motion (described below) because LAR alone can't distinguish a static open mouth from active speech articulation.

B. Face Tracking (IoU-based)

To maintain stable 'Speaker 1', 'Speaker 2' identities across frames, faces are tracked using Intersection-over-Union (IoU) matching between their bounding boxes:

$$\text{IoU} = \text{area}(\text{intersection}) / \text{area}(\text{union})$$

Each new frame's face observations are greedily matched to existing tracks (highest IoU first). A minimum IoU of 0.2 is required. Unmatched observations spawn new tracks with the next available ID. Tracks unseen for 0.5 seconds are dropped. This is simple but effective for faces that stay roughly in place (typical for video calls or seated conversations).

C. Differential Motion (the Key Innovation)

The activity signal is NOT simply 'how much the mouth moves.' That would trigger on head shaking, body movement, or any camera jitter. Instead, we measure:

$$\text{activity} = \text{lip_landmark_displacement} - \text{reference_landmark_displacement}$$

Where:

- Lip landmarks = outerLips + innerLips (the parts that move during speech)
- Reference landmarks = leftEye + rightEye + nose + noseCrest (stable during speech)

Both sets use normalized points (within the face bounding box), so head translation and scale changes are already canceled. The subtraction further cancels head rotation and jitter. What remains is pure mouth articulation.

The motion is computed as mean Euclidean displacement between the current and previous frame's landmark positions, EMA-smoothed (factor 0.4). The net value (lip minus reference, clamped to ≥ 0) becomes the 'activity' signal.

D. Hysteresis + Hold Time

To prevent the active/inactive flag from flickering between words (natural pauses in speech), two mechanisms are used:

1. Hysteresis: Different thresholds for turning ON (0.008) vs turning OFF (0.005). The speaker must exceed the higher threshold to start 'speaking' but only needs to drop below the lower threshold to stop.
2. Hold time: Once active, the flag stays on for at least 0.35 seconds even if motion stops. This bridges the brief pauses between syllables and words.

Both values are tunable via the live sidebar sliders.

E. Voice Activity Detection (VAD)

A separate energy-based VAD runs on the audio stream to answer: 'Is anyone speaking at all?' This operates independently of the visual system.

The VAD computes energy in dB from each audio buffer's mean-square value, and maintains an adaptive noise floor:

- Seeded from the first audio sample (not a fixed guess)
- Falls toward quieter ambient at rate 0.05 (tracks silence quickly)
- Rises toward louder ambient at rate 0.005 (creeps up slowly)
- Both directions always adapt (no speech gate - this prevents a deadlock where a false trigger freezes the floor)
- Clamped to $[-80, -20]$ dB

Speech is detected when energy exceeds noise floor by onMarginDB (default 6 dB), with hysteresis (offMargin 3 dB) and 0.30s hangover.

F. Cross-Modal Overlap Fusion

The final overlap decision combines both modalities:

$\text{overlapDetected} = (2+ \text{faces lip-active}) \text{ AND } (\text{audio VAD says speech})$

This means overlap requires BOTH visual evidence (multiple mouths moving) and audio evidence (sound actually present). This dramatically reduces false positives compared to using either signal alone - body movement without sound won't trigger, and ambient noise without lip motion won't trigger.

Design Decision: We deliberately chose NOT to add acoustic speaker diarization (like pyannote). Faces already provide speaker identity for free. Adding an audio diarization model would add seconds of latency and computational cost for redundant information. Audio diarization would only help for off-screen speakers (no face visible), which is out of scope for this camera-based system.

Tunable Parameters

Parameter	Default	Range	Effect
Talk threshold	0.008	0.002-0.030	Activity level to start speaking
Hold time	0.35s	0.1-1.0s	Keep active after motion stops
Speech margin	6 dB	2-15 dB	dB above noise floor for VAD
EMA smoothing	0.4	fixed	Smoothing factor for LAR + activity
IoU threshold	0.2	fixed	Min overlap to keep same speaker ID

Source Files

[LipGeometry.swift](#)

LAR computation from inner/outer lip landmarks

[LipActivityDetector.swift](#)

Face tracking, differential motion, hysteresis

[VoiceActivityDetector.swift](#)

Energy VAD with adaptive noise floor

[AudioMetrics.swift](#)

CMSampleBuffer -> mean-square energy

5. Stage 3 - Video-Guided Spectral Separation

Purpose

When two or more speakers talk simultaneously (overlap), attempt to isolate a target speaker's voice using their pitch profile. The video timeline tells us WHEN overlap occurs and WHO is speaking; the audio processing removes frequency components that don't match the target speaker.

How It Works

A. Short-Time Fourier Transform (STFT)

The STFT converts a time-domain audio signal into a time-frequency representation:

Parameters:

- FFT size: 1024 samples (64ms at 16kHz)
- Hop size: 256 samples (16ms) = 75% overlap between frames
- Window: sqrt-Hann (both analysis and synthesis)

Each frame is windowed, then transformed using `vDSP_fft_zip` (full complex FFT). This produces a Spectrogram with real and imaginary components per frequency bin per time frame.

The inverse STFT (ISTFT) reconstructs the time-domain signal via overlap-add: each frame is inverse-FFT'd, scaled by $1/N$, multiplied by the synthesis window, and added into the output buffer. A normalization array tracks the accumulated window product per sample to ensure perfect reconstruction.

Round-trip accuracy (STFT -> ISTFT with no modification): maximum error $2.4e-7$, essentially perfect reconstruction.

B. Pitch Estimation (F0)

Each speaker's fundamental frequency (F0) is estimated from their SOLO speech segments (times when only they are speaking, identified from the video timeline).

Algorithm: Normalized Autocorrelation

1. Process 1024-sample frames with 512-sample hop
2. For each frame, compute normalized cross-correlation at lags corresponding to 75-350 Hz
3. Find the lag with highest correlation (voicing clarity)
4. Use smallest lag near global maximum (avoids octave-down errors)
5. Only accept frames with voicing clarity > 0.5
6. Return median F0 across all voiced frames

The result is a single F0 estimate per speaker (e.g., 'Speaker 1: 135 Hz, Speaker 2: 225 Hz').

C. Harmonic Comb Masking

During overlap, a spectral mask is applied that keeps frequency bins near the target speaker's harmonics and attenuates everything else:

For each frequency bin k :

```
freq = k * sampleRate / fftSize
nearestHarmonic = round(freq / F0) * F0
distance = |freq - nearestHarmonic|
gain = floor + (1 - floor) * exp(-distance^2 / (2 * sigma^2))
```

Where:

- floor = 0.08 (minimum gain, prevents total silence)
- sigma = max(20, 0.18 * F0) Hz (Gaussian width, allows for pitch drift)

The mask is applied symmetrically (mirror at Nyquist) to keep the output real-valued. The real and imaginary parts of each STFT frame are multiplied by the same mask.

D. Three-Way Gating Logic

For each STFT frame, the video timeline determines the action:

1. TARGET NOT ACTIVE: Multiply all bins by 0 (silence). The target isn't speaking, so remove everything - it's another speaker's solo.
2. TARGET SOLO: Keep frame untouched. Only the target is speaking, so no separation needed.
3. OVERLAP (or 'always mask' mode): Apply the harmonic comb mask. Keep the target's pitch harmonics, attenuate the rest.

Known Limitation: Harmonic masking is a classical DSP approach with inherent limitations. It cannot separate speakers with similar pitches (e.g., two males at 130/135 Hz). It damages unvoiced consonants (s, sh, f, t) which have no harmonic structure. It may introduce musical noise artifacts. This is a demonstrator - production quality would require a neural separation model like Conv-TasNet.

Source Files

[STFTProcessor.swift](#)

Full-complex STFT/ISTFT via vDSP, sqrt-Hann windows

[PitchEstimator.swift](#)

Normalized autocorrelation F0 estimation

[SpectralGate.swift](#)

Harmonic comb mask, 3-way gating logic

[SpectrogramView.swift](#)

Log-magnitude spectrogram rendering to UIImage

[MediaLoader.swift](#)

AVAssetReader -> 16kHz mono Float32 audio

AudioPlayer.swift

Temp .caf file + AVAudioPlayer for A/B playback

6. Stage 4 - Speech Transcription (WhisperKit)

Purpose

Convert the audio signal into text using an on-device speech recognition model, with word-level timestamps that enable speaker attribution in the next stage.

How It Works

A. WhisperKit Integration

WhisperKit is an open-source Swift framework that runs OpenAI's Whisper models natively on Apple Silicon using CoreML and the Neural Engine. The integration:

- Added as an SPM dependency (pinned to `>= 0.9.0, < 1.0.0` to avoid v1.0.0 API changes)
- Resolved version: 0.18.0
- Model: `openai_whisper-small` (multilingual, 244M parameters)
- Language: forced to Indonesian ('id') via `DecodingOptions`
- Word timestamps: enabled for per-word timing

On first use, the CoreML model is downloaded from HuggingFace (~500MB). Subsequent runs use the cached model. The app is not sandboxed, so network access works without entitlements.

B. Transcription Pipeline

1. Audio is provided as [Float] at 16kHz mono (from `MediaLoader`)
2. `WhisperKit.transcribe(audioArray:decodeOptions:)` runs the full encoder-decoder pipeline
3. Results include segments (sentences) and words (individual tokens with timestamps)
4. Special tokens (`<|startoftranscript|>`, `<|id|>`, `<|3.20|>`) are stripped via regex
5. Empty segments/words after cleaning are filtered out
6. Output: `[TranscriptSegment]` each containing `[TranscriptWord]` with start/end times

C. Token Cleaning

Whisper's raw output includes special tokens in the text that are not meant for display:

```
<|startoftranscript|> <|id|> <|notimestamps|> <|3.20|>
```

These are stripped using the regex pattern: `<\[^\]*\|>`

This removes any `<|...|>` token, leaving only the actual transcribed text.

D. Model Selection

The system was tested with two Whisper model sizes:

whisper-base (74M parameters): Fast but poor Indonesian accuracy. Many hallucinations and incorrect words.

whisper-small (244M parameters): Much better Indonesian transcription. Currently the default. Good balance of speed and accuracy for on-device use.

For production use, a fine-tuned Indonesian model (e.g., whisper-small-id trained on Indonesian speech data, converted to CoreML via whisperkittools) would significantly improve accuracy.

Source Files

[Transcriber.swift](#)

WhisperKit wrapper, model loading, token cleanup

[project.yml](#)

WhisperKit SPM dependency declaration

7. Stage 5 - Speaker Attribution

Purpose

Assign each transcribed word to the face that was most actively speaking at that moment, then merge consecutive same-speaker words into utterances. This is the final step that produces the speaker-attributed transcript.

How It Works

A. Word-to-Face Mapping

For each transcribed word (with its start/end timestamps from WhisperKit):

1. Compute the word's midpoint: $\text{mid} = (\text{word.start} + \text{word.end}) / 2$
2. Look up the video timeline at that moment: `timeline.dominantSpeaker(at: mid)`
3. `dominantSpeaker` finds the nearest video frame, filters for faces with activity above the threshold, and returns the face ID with the highest activity value (`argmax`)
4. If no face is active above threshold, the word is attributed to 'unknown' (?)

This approach works because WhisperKit provides word-level timestamps, and the video timeline records per-frame per-face activity values. By looking up the most-active face at each word's midpoint, we get a natural speaker assignment.

B. Utterance Merging

After individual word attribution, consecutive words assigned to the same speaker are merged into utterances:

Word: 'Ini' -> Speaker 1 (3.9s)
Word: 'adalah' -> Speaker 1 (4.1s) -> merged
Word: 'contoh' -> Speaker 1 (4.3s) -> merged
Word: 'Itu' -> Speaker 2 (5.0s) -> new utterance

Result:

[Speaker 1] 'Ini adalah contoh' (3.9 - 4.5s)
[Speaker 2] 'Itu ...' (5.0s - ...)

Each `AttributedUtterance` stores: speaker ID, merged text, start time, and end time.

C. Attribution Accuracy

The accuracy of speaker attribution is bounded by:

1. Video active-speaker detection quality: If the lip motion detector wrongly flags a face as active or misses the true speaker, the word will be misattributed.

2. WhisperKit word timestamp accuracy: Whisper's word boundaries are approximate (typically within ~200ms). For fast speaker turns, this may misalign.

3. Overlap sensitivity threshold: The same slider controls both overlap detection and attribution. A lower threshold means more faces qualify as 'active', making it harder to pick the dominant one.

In practice, attribution works well for clear turn-taking conversations and degrades during fast back-and-forth or genuine overlapping speech.

Source Files

[Attribution.swift](#)

Word -> face mapping + utterance merging

[VideoAnalyzer.swift](#)

Timeline.dominantSpeaker(at:threshold:)

8. Offline Analysis Lab (Analyze File Mode)

Purpose

The analysis lab runs the full pipeline offline on a recorded video file, letting you evaluate each stage's output, tune parameters, and compare before/after spectral separation. This is essential for development and debugging since real-time evaluation of spectral quality is impractical.

Features

- File import: Load any .mov, .mp4, or audio file via the system file picker
- Video analysis: Runs face detection + lip activity on every video frame with progress %
- Speaker cards: Face thumbnail, speaker ID, and estimated F0 per detected speaker
- Overlap sensitivity slider: Re-derives the active/overlap timeline instantly (no re-decode)
- 'Mask whenever active' toggle: Forces harmonic masking even outside overlap regions
- Close-pitch warning: Alerts when two speakers have F0 within 20 Hz (unseparable)
- Before/after spectrograms: Original vs. separated, log-magnitude heatmap
- A/B playback: Play original or separated audio (via temp .caf + AVAudioPlayer)
- Transcription: WhisperKit transcribe button with model loading status
- Speaker-attributed transcript: Colored 'Speaker N' chips per utterance line

Workflow

1. Switch to 'Analyze File' tab (camera stops automatically)
2. Click 'Load video...' and select a recording
3. Wait for audio extraction and video analysis (progress bar shows %)
4. Review speaker cards - check that faces are detected and F0 estimates look reasonable
5. Adjust overlap sensitivity slider - watch the 'overlap Xs' readout change
6. Select a target speaker to separate
7. Compare original and separated spectrograms visually
8. Use A/B playback to listen to the difference
9. Click 'Transcribe' to run WhisperKit and see the speaker-attributed transcript

Source Files

[AnalysisView.swift](#)

Full analysis UI + AnalysisViewModel orchestration

[VideoAnalyzer.swift](#)

Offline video -> Timeline (raw per-frame activity)

[ContentView.swift](#)

Live/Analyze mode switch, camera lifecycle

9. Current Limitations & Known Issues

Spectral Separation Quality

- Cannot separate speakers with similar F0 (e.g., two males at 130/135 Hz). Harmonic comb masking fundamentally requires distinct pitches.
- Damages unvoiced consonants (s, sh, f, t, k) because they have no harmonic structure - the comb mask attenuates them as 'non-target'.
- May introduce musical noise artifacts at mask boundaries.
- Only works for voiced speech segments - whispered or breathy speech has no clear harmonics to target.

Single Microphone Limitation

- With one microphone, audio energy cannot be attributed to a specific face. If two people both move their lips while audio is present, both read as active.
- Cross-modal fusion reduces this (requires BOTH lip motion AND audio), but cannot fully resolve who produced which sound from a single channel.

Active Speaker Detection

- Requires faces to be visible and roughly front-facing. Profile views or occluded faces may not be detected by Vision.
- Rapid head movement can briefly confuse the differential motion measure, though the reference-landmark subtraction handles most cases.
- The IoU face tracker is simple - it can lose identity if faces cross paths or temporarily leave the frame.

Transcription

- Using stock openai_whisper-small, not fine-tuned for Indonesian. Accuracy is good but not production-grade for specialized vocabulary.
- First transcription requires downloading the CoreML model (~500MB). Needs network connectivity.
- Transcription + attribution currently only works in offline (Analyze File) mode. Live real-time transcription is not yet implemented.

Platform

- macOS only (14.0+). Uses AVCaptureSession APIs not available on iOS in the same form.
- Not sandboxed - suitable for development but needs entitlements for App Store.
- No code signing (identity '-') - for local development only.

10. Improvement Opportunities

High Impact

1. Neural Source Separation (Conv-TasNet / SepFormer)

Replace harmonic comb masking with a learned neural separation model. Conv-TasNet or SepFormer can separate overlapping speakers regardless of pitch similarity, handling unvoiced consonants and same-gender speakers. The model would be converted to CoreML for on-device inference.

Effort: High (model training/conversion, CoreML optimization)

Impact: Dramatically better separation quality - the single biggest improvement possible.

2. Fine-Tuned Indonesian Whisper Model

Fine-tune whisper-small (or whisper-medium) on Indonesian speech data, then convert to CoreML using the whisperkittools Python package. This would improve transcription accuracy significantly, especially for Indonesian-specific vocabulary, names, and colloquialisms.

Effort: Medium (dataset collection + fine-tuning + CoreML conversion)

Impact: Much better transcription accuracy for the target language.

3. Live Real-Time Transcription + Attribution

Bring WhisperKit streaming transcription into the live mode. Buffer audio in sliding windows (e.g., 5-second chunks with 1-second overlap), transcribe incrementally, and attribute each word to the currently active face in real-time.

Effort: Medium (buffering logic, incremental updates, UI streaming)

Impact: Completes the live pipeline vision - real-time speaker-attributed captions.

Medium Impact

4. SpeakerKit Diarization (Off-Screen Speakers)

Integrate Argmax's SpeakerKit (now in argmax-oss-swift v1.0.0) for audio-based speaker diarization. This would handle speakers who are off-camera or whose faces are not detected. The visual and audio identities could be fused for robust speaker tracking.

Effort: Medium

Impact: Handles off-screen speakers; more robust speaker identification.

5. Audio-Visual Lip Sync Correlation

Compute per-face correlation between lip motion timing and audio energy/onset patterns. This would tell you which face is actually producing the heard audio, solving the ambiguity when multiple faces move their lips during the same audio.

Effort: Medium (signal processing + correlation analysis)

Impact: More accurate active-speaker detection in multi-speaker scenes.

6. Improved Face Tracking

Replace IoU-based tracking with a more robust multi-object tracker (e.g., Deep SORT with face embeddings, or Apple's VNTrackObjectRequest). This would maintain identity through temporary occlusions, face crossings, and frame exits/re-entries.

Effort: Medium

Impact: More stable Speaker IDs across longer recordings.

Lower Effort / Nice to Have

7. Larger Whisper Models

Switch from whisper-small to whisper-medium or whisper-large for better accuracy. Trade-off is inference speed and memory. Test on your hardware to find the sweet spot.

Effort: Low (just change the model name)

Impact: Better transcription, slower inference.

8. Confidence-Based Attribution

Instead of hard argmax attribution (assign to the single most-active face), use a confidence score. When the top two faces have similar activity levels, mark the word as 'uncertain' rather than guessing. Show confidence in the UI.

Effort: Low

Impact: More honest/transparent attribution.

9. Export Functionality

Add export options: SRT/VTT subtitle files, plain text transcript, CSV with timestamps and speakers, or separated audio WAV files. Useful for downstream processing.

Effort: Low

Impact: Practical utility for end users.

11. Performance Evaluation & Comparison Ideas

What to Measure

To evaluate and improve the system, you need ground truth annotations and quantitative metrics. Here are practical approaches for each stage:

A. Active Speaker Detection Accuracy

Ground truth: Manually annotate a test video with who is speaking at each moment (per-frame or per-100ms labels).

Metrics:

- Frame-level accuracy: % of frames where the active/inactive prediction matches ground truth
- Precision: Of frames predicted 'active', how many are truly active?
- Recall: Of truly active frames, how many are detected?
- F1 score: Harmonic mean of precision and recall
- Overlap detection rate: What % of true overlap segments are correctly detected?

Comparison:

- Compare LAR-only detection vs. differential motion detection
- Compare visual-only overlap vs. cross-modal (visual + audio VAD) fusion
- Sweep threshold values and plot precision-recall curves

B. Spectral Separation Quality

Ground truth: Record each speaker separately (isolated channels), then mix them. The individual recordings are the ground truth for separation.

Metrics:

- SDR (Signal-to-Distortion Ratio): Overall separation quality in dB
- SIR (Signal-to-Interference Ratio): How well the other speaker is suppressed
- SAR (Signal-to-Artifacts Ratio): How much artifact (musical noise) is introduced
- PESQ/POLQA: Perceptual speech quality measures

You can compute SDR/SIR/SAR using the Python library 'mir_eval' or 'museval':

```
# Python example with mir_eval
import mir_eval
sdr, sir, sar, _ = mir_eval.separation.bss_eval_sources(
    reference_sources, # [2, num_samples] ground truth
    estimated_sources  # [2, num_samples] your separation output
)
```

Comparison:

- Harmonic comb masking (current) vs. no separation (baseline)

- Harmonic comb masking vs. neural separation (Conv-TasNet, if implemented)
- Different F0 estimation methods
- Different mask floor values (0.05, 0.08, 0.15)
- Different Gaussian sigma widths

C. Transcription Accuracy

Ground truth: Human transcription of the test audio.

Metrics:

- WER (Word Error Rate): (substitutions + insertions + deletions) / total reference words
- CER (Character Error Rate): Same formula at character level (better for Indonesian)

You can compute WER using the Python library 'jiwer':

```
# Python example with jiwer
from jiwer import wer, cer
reference = 'ini adalah contoh kalimat'
hypothesis = 'ini dalah contoh kalimat'
print(f'WER: {wer(reference, hypothesis):.2%}')
print(f'CER: {cer(reference, hypothesis):.2%}')
```

Comparison:

- whisper-base vs. whisper-small vs. whisper-medium (accuracy vs. speed trade-off)
- Stock model vs. fine-tuned Indonesian model
- Original audio vs. separated audio (does separation improve or hurt transcription?)
- Clean speech vs. overlapping speech (where does the model struggle?)

D. Speaker Attribution Accuracy

Ground truth: Manually label which speaker said each word/utterance.

Metrics:

- DER (Diarization Error Rate): Standard metric for speaker diarization
$$\text{DER} = (\text{missed speech} + \text{false alarm} + \text{speaker confusion}) / \text{total speech duration}$$
- Word-level speaker accuracy: % of words attributed to the correct speaker
- Utterance-level speaker accuracy: % of utterances with correct speaker label

Comparison:

- Video-only attribution (current) vs. audio diarization (e.g., SpeakerKit)
- Different overlap sensitivity thresholds
- Different attribution strategies (argmax vs. weighted vs. time-window voting)

E. End-to-End Benchmark

The most meaningful evaluation is end-to-end: given a test video with multiple speakers, how well does the system produce a correct speaker-attributed transcript?

Suggested test set:

1. Two speakers with distinct pitches, clear turn-taking
2. Two speakers with similar pitches, clear turn-taking
3. Two speakers with some overlapping speech
4. Three or more speakers
5. Varying recording conditions (quiet room, noisy environment)

For each clip, measure WER per speaker and overall DER.

Practical Comparison Table

Here is a template for comparing different configurations:

Configuration	WER (%)	DER (%)	SDR (dB)	Latency	Notes
whisper-base, no sep.			N/A		Baseline
whisper-small, no sep.			N/A		Current
whisper-small + comb					Current
whisper-small + neural					Future
whisper-medium, no sep.			N/A		Larger model
fine-tuned ID, no sep.			N/A		Future

(Fill in with your test results)

12. Source File Reference

Application Core

[LiveCaptionApp.swift](#)

@main entry point, WindowGroup (min 760x560)

[ContentView.swift](#)

Live/Analyze tab switch, CaptureManager lifecycle

[project.yml](#)

XcodeGen spec: targets, dependencies, build settings

Stage 1: Capture & Detection

[CaptureManager.swift](#)

AVCaptureSession (video+audio), delegates, cross-modal fusion

[FaceLandmarkProcessor.swift](#)

VNDetectFaceLandmarksRequest Rev3 wrapper

[CameraPreview.swift](#)

NSViewRepresentable, overlay rendering (CAShapeLayer)

[AudioMetrics.swift](#)

CMSampleBuffer -> mean-square (Float32/Int16, vDSP)

Stage 2: Active Speaker Detection

[LipGeometry.swift](#)

LAR = inner vertical / outer horizontal (scale-invariant)

[LipActivityDetector.swift](#)

IoU tracking, differential motion, hysteresis, hold time

[VoiceActivityDetector.swift](#)

Energy VAD, adaptive noise floor, hangover

Stage 3: Spectral Separation

[STFTProcessor.swift](#)

vDSP full-complex FFT, sqrt-Hann, overlap-add inverse

[PitchEstimator.swift](#)

Normalized autocorrelation, octave-bias, median F0

[SpectralGate.swift](#)

Gaussian harmonic comb mask, 3-way gating

[SpectrogramView.swift](#)

Log-magnitude heatmap (black-purple-orange-yellow)

[MediaLoader.swift](#)

AVAssetReader -> 16kHz mono Float32

[AudioPlayer.swift](#)

Temp .caf + AVAudioPlayer for playback

Stage 4: Transcription

[Transcriber.swift](#)

WhisperKit wrapper, DecodingOptions, token cleanup regex

Stage 5: Attribution

[Attribution.swift](#)

Word midpoint -> dominantSpeaker, utterance merging

Offline Analysis

[VideoAnalyzer.swift](#)

Batch video -> Timeline (raw per-frame activity, thumbnails)

[AnalysisView.swift](#)

Full analysis lab UI + AnalysisViewModel